

EXPERIMENT NO. 4

Count Equal Pairs with Index Product Divisible by k

Programming Language: C

Name : Bodham susma

Register No. : 192571065

Aim

To write a C program that counts the pairs of indices holding equal values whose index product is divisible by k .

Question

Given a 0-indexed integer array `nums` of length n and an integer k , return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by k .

Example 1

Input: `nums = [3,1,2,2,2,1,3]`, $k = 2$

Output: 4

Explanation: There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 == 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 == 12$, which is divisible by 2.

Example 2

Input: `nums = [1,2,3,4]`, $k = 1$

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i,j) that meet all the requirements.

Algorithm

Step 1: Start.

Step 2: Store the array `nums` and the integer k , compute n , and set `pairCount = 0`.

Step 3: For each index i from 0 to $n - 1$, loop over every index j from $i + 1$ to $n - 1$.

Step 4: If `nums[i] == nums[j]` and $(i * j) \% k == 0$, increment `pairCount`.

Step 5: After all pairs are checked, print `pairCount`.

Step 6: Stop.

Program (C)

```
#include <stdio.h>

int main() {
    // Example 1 Input (Modify these to test other cases)
    int nums[] = {3, 1, 2, 2, 2, 1, 3};
    int n = sizeof(nums) / sizeof(nums[0]);
    int k = 2;

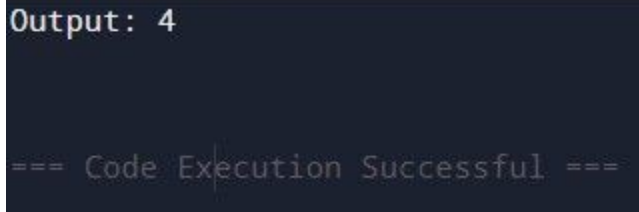
    int pairCount = 0;

    // Outer loop: selects the first index 'i'
    for (int i = 0; i < n; i++) {
        // Inner loop: selects the second index 'j' (must be strictly greater than 'i')
        for (int j = i + 1; j < n; j++) {
            // Condition 1: The elements at index i and j are equal
            // Condition 2: The product of the indices (i * j) is divisible by k
            if (nums[i] == nums[j] && (i * j) % k == 0) {
                pairCount++;
            }
        }
    }

    // Output the total number of valid pairs
    printf("Output: %d\n", pairCount);

    return 0;
}
```

Output



```
Output: 4
```

```
=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result

Thus the program was executed successfully and the output was verified.